DEVIXO SOLUTIONS

Student Record Management System

C++ Programming Internship
TASK 01 — PROJECT REPORT
Submitted By: Ayesha Raheela
Internship Domain: C++ Programming
Organization: Devixo Solutions
Project Type: Console-Based Application
Programming Language: C++
Submission Date: October 2026

Project Documentation and Implementation Report

Devixo Solutions
Internship

Table of Contents

1 Introduction

2 Project Objective

3 Problem Statement

4 Project Scope and Features

5 Technologies Used

6 Implementation Methodology

7 Program Structure and Data Representation

8 Implementation and Code Explanation

9 File Handling and Data Persistence

10 Input Validation and Error Handling

11 Testing and Results

12 Challenges Faced and Solutions

13 Learning Outcomes

14 Conclusion

15 GitHub Repository

Introduction

The Student Record Management System is a console-based application developed in C++ as part of Devixo Solutions Internship Task 01.

The purpose of this project is to demonstrate the practical application of fundamental programming concepts by developing a system that manages student information through a menu-driven interface.
The application allows users to add new student records, view existing records, search for a student by ID, update student information, and delete records when required. It also incorporates file handling to save student information and load previously saved records when the program starts.
The project applies important C++ concepts, including structures, arrays, functions, loops, conditional statements, strings, input validation, and file input/output operations.
This project provides practical experience in designing a structured program, organizing code into functions, handling user input, and testing different program operations.

Inclusivity

Transparency

Accountability

Project Objective

The primary objective of this project is to develop a functional Student Record Management System using C++.

The specific objectives are:

- To create a user-friendly, menu-driven console application.
- To implement operations for adding, viewing, searching, updating, and deleting student records.
- To use structures and arrays to organize student information.
- To divide the program into functions for better organization and readability.
- To implement file handling for storing and retrieving student records.
- To validate user input and prevent common data-entry errors.
- To test the application under different input conditions and verify its functionality.



PROBLEM STATEMENT AND SCOPE

Problem Statement

Managing student information requires an organized way to record, retrieve, modify, and remove student details.
Without a suitable record-management system, these operations may become inconvenient and can lead to inconsistent or incorrect information.
The objective of this project is to address this problem by developing a simple C++ application that allows users to manage student records efficiently through a console menu.

Each student record contains three main fields:

- Student ID: A unique identifier for each student.
- Student Name: The name of the student.
- Marks: The student's marks, restricted to the range from 0 to 100.

The application also provides input validation to reject invalid IDs, duplicate IDs, invalid numeric input, and marks outside the permitted range.



Project Scope and Features

The application includes the following features:

1. Add Student
Allows the user to enter a student's ID, name, and marks. The program checks the validity of the information before adding the record.
2. View Students
Displays the available student records in a formatted table.
3. Search Student
Searches for a student using their ID and displays the matching information if the student exists.
4. Update Student
Allows the user to modify an existing student's name and marks.
5. Delete Student
Removes a student record using the student's ID. The remaining records are shifted within the array to maintain the list.
6. Save and Load Records
Uses a text file named students.txt to store records and retrieve them when the application starts.
7. Input Validation
Checks numeric input, duplicate student IDs, positive IDs, and valid marks.
8. Error Messages
Displays suitable messages when input is invalid or a requested student record cannot be found.

TECHNOLOGIES USED

The project was developed using the following technologies and C++ programming concepts.

1

Programming Language

C++
C++ was used to implement the application logic, manage student records, process user input, and perform file operations.

2

Development Environment

CodeUtility Online C++ Compiler
The program was developed and tested in an online C++ environment that provided a source-code editor, execution console, and workspace for project files.

3

Header Files and Libraries

The program uses the following standard C++ headers:

- iostream — for console input and output.
- string — for storing and processing student names.
- fstream — for reading and writing files.
- sstream — for processing text-based record data.
- limits — for managing input-stream errors.
- iomanip — for formatting tabular output.

4

Programming Concepts

The implementation demonstrates:

- Structures for representing student records.
- Arrays for storing multiple student records.
- Functions for separating program operations.
- Loops and conditional statements for menu processing.
- Strings for handling student names.
- Input validation for checking user-provided data.
- File handling for persistent record storage.

These concepts help organize the program and make its functionality easier to understand and maintain.

IMPLEMENTATION METHODOLOGY

The application was developed in stages to ensure that each major operation could be implemented and tested independently.

Step 1: Define the Student Structure

A structure named Student represents an individual student record. It contains the student's ID, name, and marks.

Step 2: Create the Record Storage

An array of Student structures is used to hold multiple student records in memory. A record counter tracks the number of stored students, and a constant defines the maximum array capacity.

Step 3: Develop Input Functions

Functions are used to read numeric input and validate marks. Invalid numeric input is handled to prevent the program from processing incorrect values.

Step 4: Implement Record Operations

Separate functions implement adding, viewing, searching, updating, and deleting student records. This functional organization makes the program easier to test and maintain.

Step 5: Implement File Handling

File-handling functions save records to students.txt and load available records when the program starts.

Step 6: Develop the Menu System

The main function displays the menu and processes the user's selected operation until the exit option is selected.

Step 7: Test the Application

The program was tested using valid input, invalid input, duplicate IDs, nonexistent student IDs, record modification, deletion, and file persistence.

The implementation approach helped identify issues during testing and verify the behavior of the main features.

PROGRAM STRUCTURE AND DATA REPRESENTATION

The application uses a structured design in which each function performs a specific task

.

1. **Student Structure**

The Student structure stores three fields:

- id — an integer representing the student ID.
- name — a string containing the student's name.
- marks — a floating-point value representing the student's marks.

2. **Array-Based Storage**

The program uses an array of Student structures to store records in memory. A record counter tracks the current number of students.
The array approach is suitable for this introductory project because it demonstrates how multiple related records can be organized using structures.

3. **Function-Based Organization**

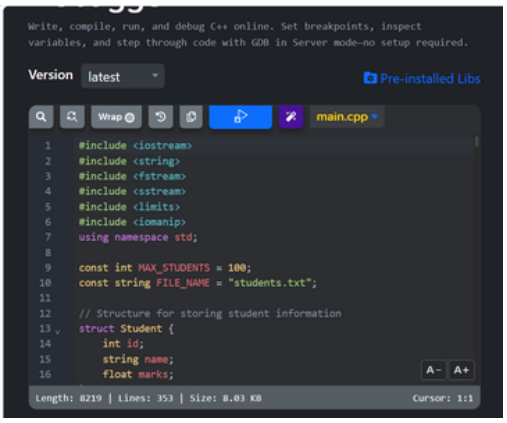
The program contains functions for input validation, record searching, menu display, file handling, and individual record operations.
This design separates responsibilities and reduces the need to place all application logic inside the main() function.

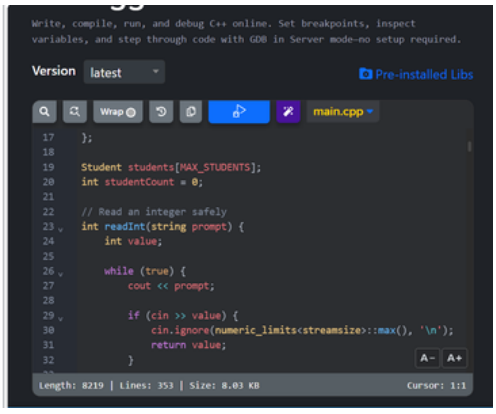
4. **Menu-Driven Interface**

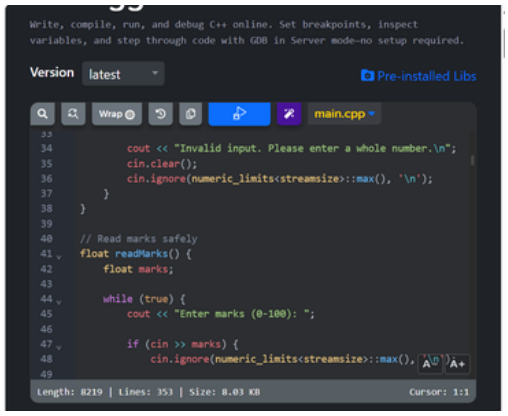
The menu provides the user with options to perform the available operations. The main loop processes the selected option and returns to the menu until the user chooses to exit.

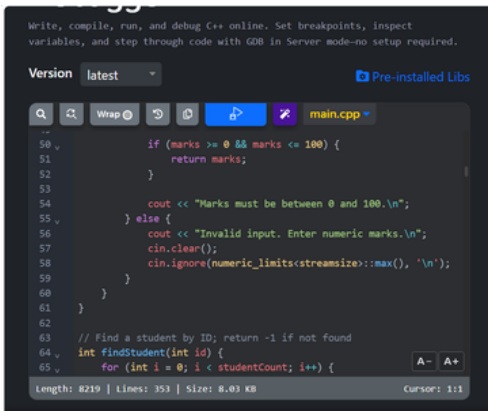
Student Structure and Arrays

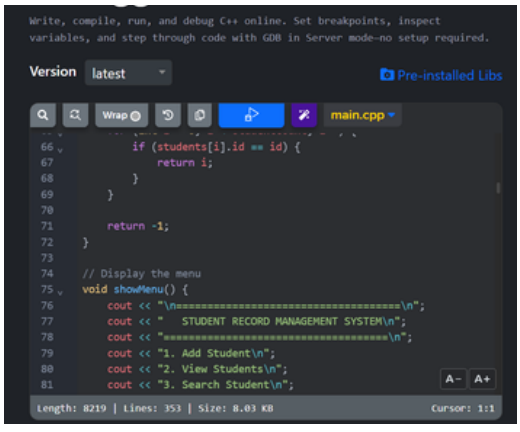
Code Screenshots:

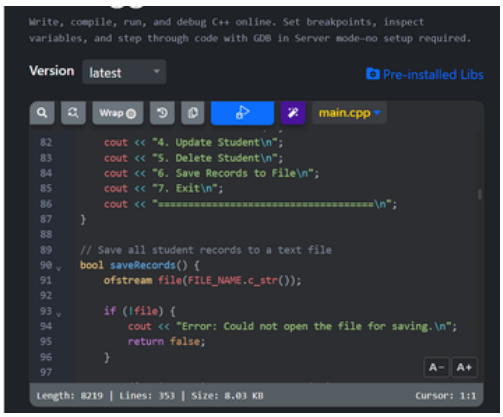






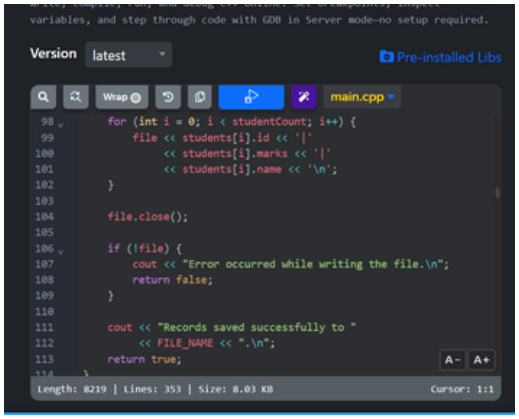


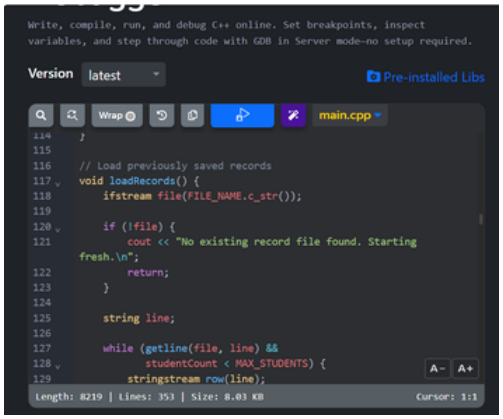


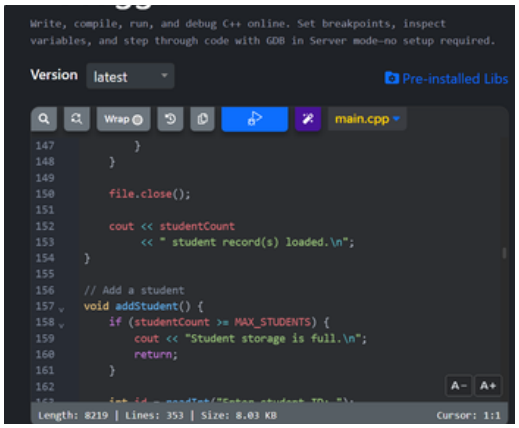


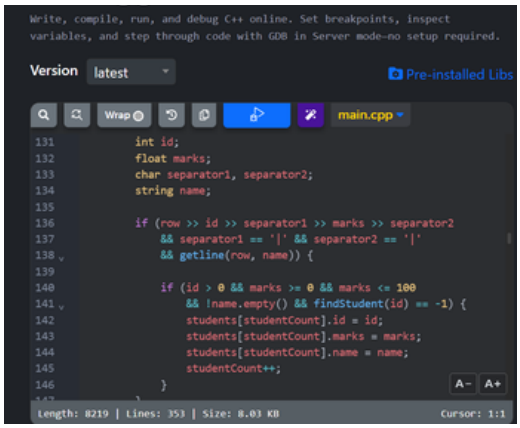
Management System Menu

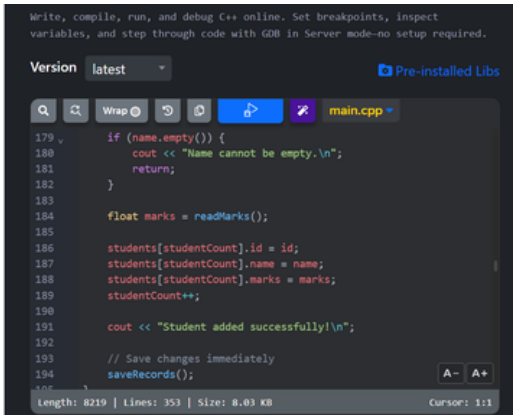
Code Screenshots:

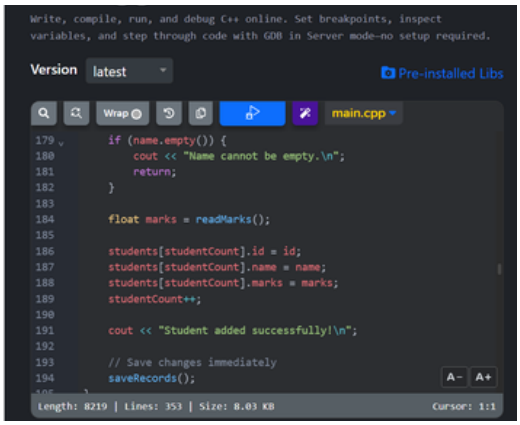






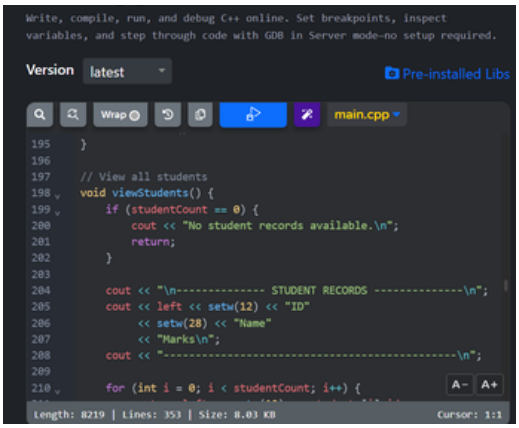


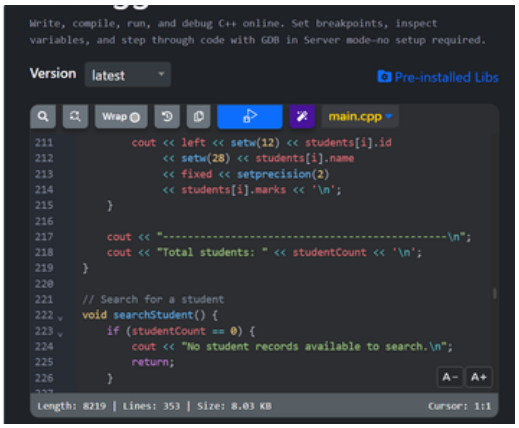


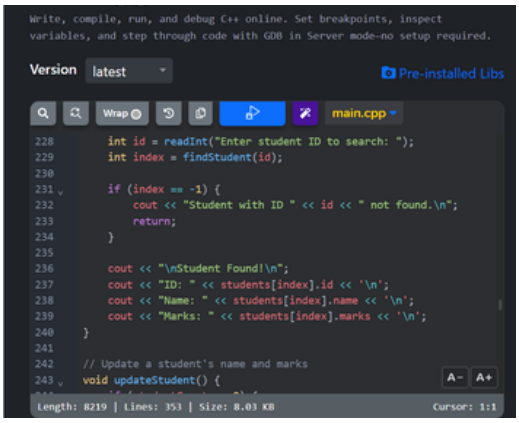


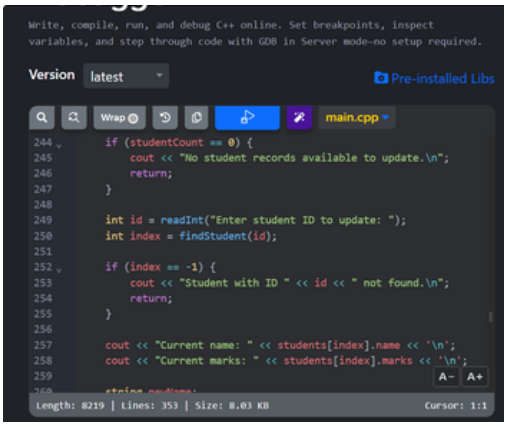
File Handling

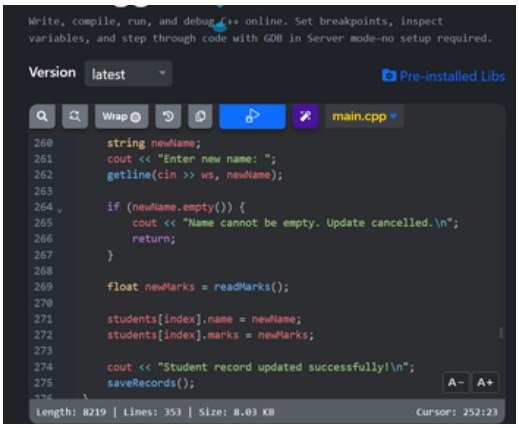
Code Screenshots:

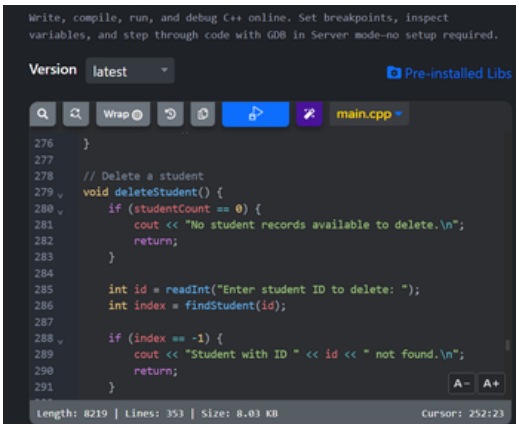






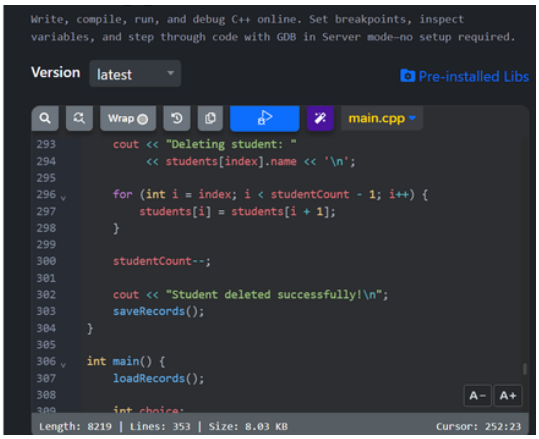




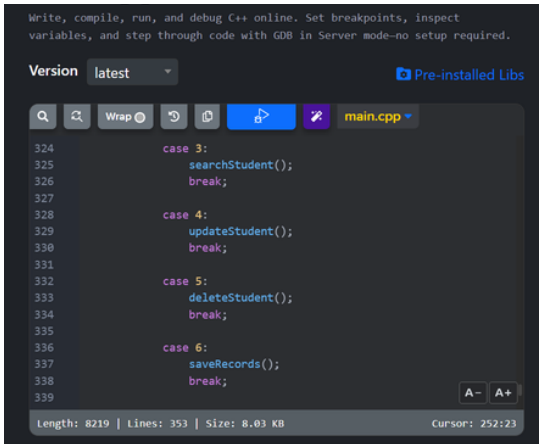


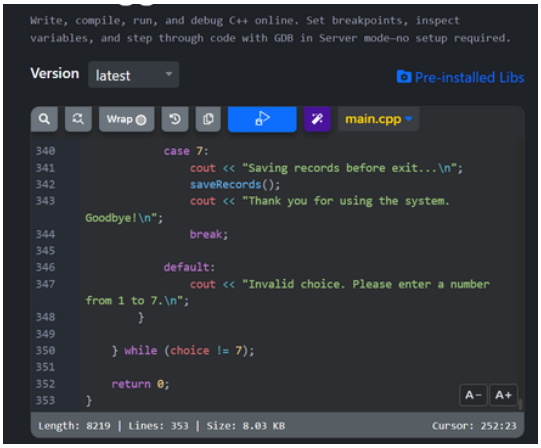
Record Operations

Code Screenshots:







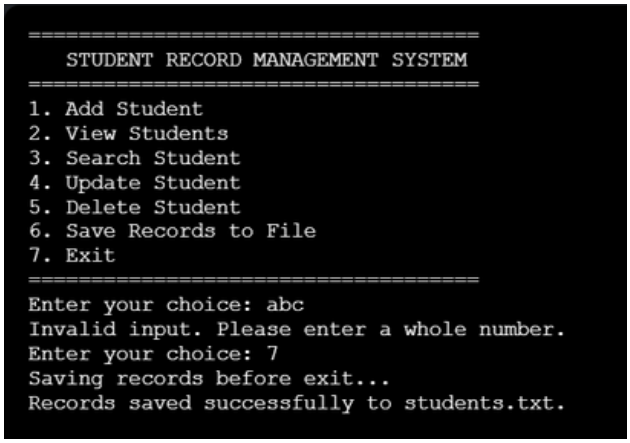


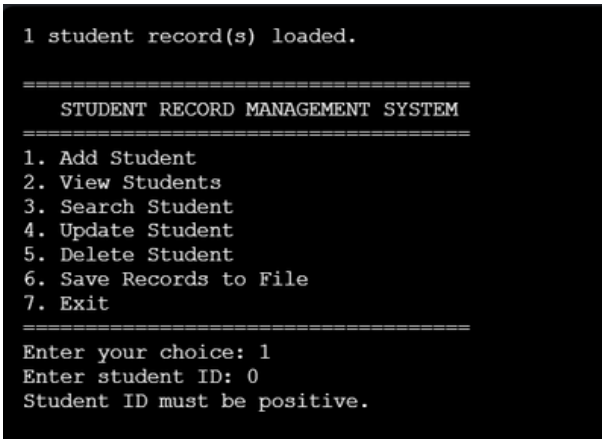
IMPLEMENTATION AND CODE EXPLANATION

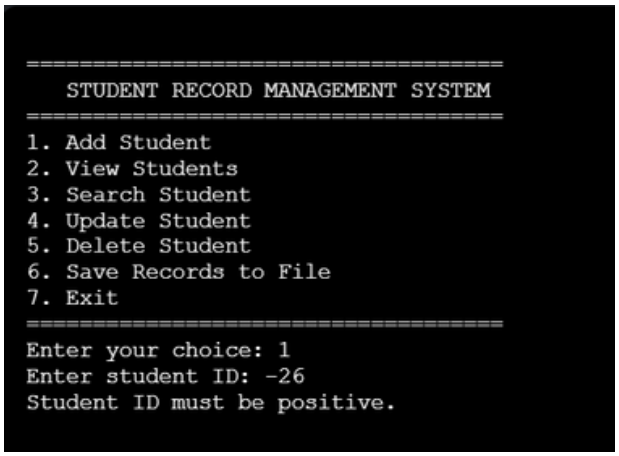
The program is divided into functions to improve readability, organization, and maintainability.

1. **Input Validation Functions**

The readInt() function reads integer input and handles invalid numeric entries. The readMarks() function ensures that student marks fall within the permitted range of 0 to 100.







IMPLEMENTATION AND CODE EXPLANATION

2. Add Student Function

The addStudent() function collects the student's ID, name, and marks. It checks the available array capacity, ensures that the ID is positive and unique, and validates the remaining information before storing the record.

```
1 student record(s) loaded.

==================================
   STUDENT RECORD MANAGEMENT SYSTEM
==================================
1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Delete Student
6. Save Records to File
7. Exit
==================================
Enter your choice: 1
Enter student ID: 102
Enter student name: Zeeshan
```

3. View Students Function

The viewStudents() function displays all available records in a formatted table. If no records are available, the program displays an appropriate message.

```
1 student record(s) loaded.

==================================
   STUDENT RECORD MANAGEMENT SYSTEM
==================================
1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Delete Student
6. Save Records to File
7. Exit
==================================
Enter your choice: 2
```

```
------------- STUDENT RECORDS -------------
ID          Name                        Marks
-------------------------------------------
101         Sara Khan                   90.00
-------------------------------------------
Total students: 1

==================================
   STUDENT RECORD MANAGEMENT SYSTEM
==================================
1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Delete Student
```

IMPLEMENTATION AND CODE EXPLANATION

4. **Search and Update Functions**

The findStudent() function searches for a record using its ID. The search and update operations use the matching record to display or modify student information.

```
1 student record(s) loaded.

==================================
   STUDENT RECORD MANAGEMENT SYSTEM
==================================
1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Delete Student
6. Save Records to File
7. Exit
==================================
Enter your choice: 1
Enter student ID: 101
A student with this ID already exists.
```

5. **Delete Student Function**

The deleteStudent() function locates the requested student and removes the record if it exists. The remaining array elements are shifted to maintain the sequence of stored records.

```
1 student record(s) loaded.

==================================
   STUDENT RECORD MANAGEMENT SYSTEM
==================================
1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Delete Student
6. Save Records to File
7. Exit
==================================
Enter your choice: 5
Enter student ID to delete: 105
Student with ID 105 not found.
```

FILE HANDLING AND DATA PERSISTENCE

File handling is an important feature of the Student Record Management System because it allows records to be stored outside the program's temporary memory.

The application uses a text file named students.txt for record storage.

1. **Saving Records**

The saveRecords() function writes student records to the text file. The records are stored using a delimiter-based format containing the student ID, marks, and name.

The file format allows the program to separate individual fields when retrieving the records.

2. **Loading Records**

The loadRecords() function reads the available records from the text file when the application starts. The program reconstructs the student records in memory so that previously saved information can be accessed.

3. **Verifying Persistence**

During testing, a student record was saved to the workspace file. In a subsequent execution, the program reported that one student record had been loaded, and the record was displayed successfully.

This confirmed that the save-and-load functionality worked in the tested persistent workspace.

4. **Importance of File Handling**

File handling makes the application more useful than a program that keeps records only in memory. It allows data to remain available between executions, provided the file remains accessible and the save operation succeeds.

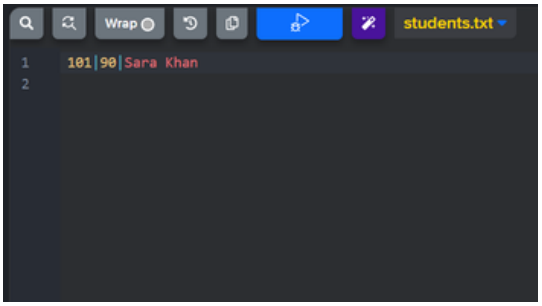
FILE HANDLING AND DATA PERSISTENCE

```
1 student record(s) loaded.

==================================
   STUDENT RECORD MANAGEMENT SYSTEM
==================================
1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Delete Student
6. Save Records to File
7. Exit
==================================
Enter your choice: 2
```

```
------------- STUDENT RECORDS -------------
ID          Name                      Marks
-------------------------------------------
101         Sara Khan                 90.00
-------------------------------------------
Total students: 1

==================================
   STUDENT RECORD MANAGEMENT SYSTEM
==================================
1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Delete Student
```



Explanation: This code section demonstrates how the application writes student records to a text file and reads them back into the program.

INPUT VALIDATION AND ERROR HANDLING

Input validation helps ensure that the program receives acceptable data and responds appropriately to common errors.

1. **Invalid Numeric Input**

The program detects invalid integer input, such as entering letters where a numeric menu choice is required. It displays an error message and requests valid input.

2. **Student ID Validation**

Student IDs must be positive. The program rejects zero and negative IDs.

3. **Duplicate ID Prevention**

The program checks whether a student ID already exists before adding a new record. If a duplicate is detected, the record is not added.

4. **Marks Validation**

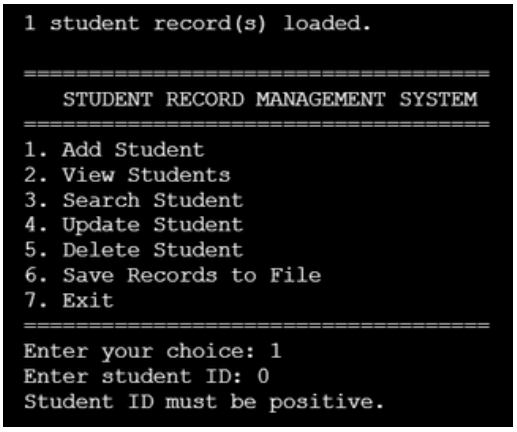
Marks must fall within the range of 0 to 100. Values outside this range are rejected.

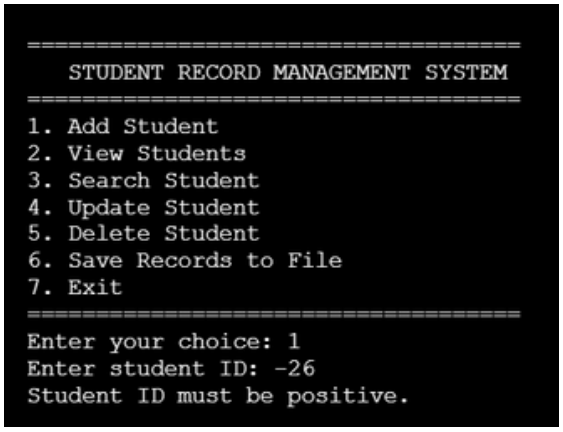
5. **Nonexistent Records**

When a user searches for or attempts to delete a student ID that does not exist, the program displays an appropriate message rather than performing the requested operation.

6. **Record Capacity**

The program checks the array capacity before adding a new student. This helps prevent writing beyond the available storage.





Explanation: These tests demonstrate that the application rejects several invalid inputs and provides informative error messages.

TESTING AND RESULTS

The application was tested through a series of functional and validation tests. The following table summarizes the observed results.

Test case	Expected Behavior	Observed Result
Add a Student	Store valid student information	Passed
View student	Display available records	Passed
Search for a student	Display the matching record	Passed
Update a student	Display the updated information	Passed
Delete an existing ID	Remove the selected record	Passed
Reject duplicate ID	Prevent duplicate record creation	Passed
Reject invalid ID	Reject zero and negative IDs	Passed
Reject invalid marks	Reject marks outside 0–100	Passed
Invalid menu input	Display an input error	Passed
Search or delete missing ID	Display a not-found message	Passed
Save and load records	Retrieve previously saved data	Passed in the test

CHALLENGES FACED AND SOLUTIONS

During the development and testing of the project, several practical challenges were encountered:

1. **Interactive Input Handling**
   - **Challenge**: The program requires multiple inputs while performing operations such as adding, updating, and deleting records. Invalid numeric input can interfere with normal input processing.
   - **Solution**: Input-validation logic was implemented to detect invalid numeric entries and request valid input.
2. **File Persistence in the Online Environment**
   - **Challenge**: File persistence depends on the online compiler's workspace and whether files remain available between executions.
   - **Solution**: A persistent workspace was used, and students.txt was checked to verify that saved records could be loaded during a subsequent execution.
3. **Testing Menu-Driven Operations**
   - **Challenge**: Each menu operation requires the appropriate sequence of inputs. If the exit option is not supplied during automated input testing, the program may continue waiting for further input.
   - **Solution**: Test inputs were organized to include the required values and an exit selection when a test was intended to finish the program.

CHALLENGES FACED AND SOLUTIONS

4. **Validating Student Information**

- **Challenge**: The application must prevent invalid IDs, duplicate IDs, and marks outside the permitted range.
- **Solution**: Validation checks were added before accepting or modifying student information.

5. **Learning Outcomes from Debugging**

- Working through these challenges improved my understanding of input-stream handling, program flow, file operations, and systematic testing.
- The debugging process also highlighted the importance of verifying program behavior using actual test results rather than relying only on code inspection.

LEARNING OUTCOMES

This project provided practical experience in applying fundamental C++ programming concepts to a functional application.
The main learning outcomes include:

1. **Understanding Structures and Arrays**

I learned how to represent related student information using a structure and organize multiple records in an array.

2. **Developing Modular Programs**

Using separate functions helped me understand how to divide a program into smaller operations that are easier to read, test, and maintain.

3. **Implementing File Handling**

I gained practical experience with reading from and writing to a text file to preserve student records between executions.

4. **Handling Invalid Input**

The project demonstrated the importance of checking user input, rejecting invalid data, and displaying useful error messages.

5. **Testing and Debugging**

Testing the application's individual features helped me identify issues and verify the expected behavior of the program.

6. **Improving Problem-Solving Skills**

Implementing searching, updating, and deleting operations strengthened my understanding of algorithms and data manipulation.

Overall, the project connected theoretical C++ concepts with practical software development and improved my confidence in building console-based applications.

CONCLUSION

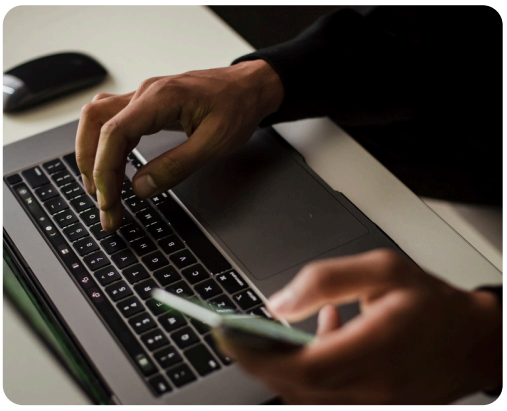
The Student Record Management System was developed as part of Devixo Solutions Internship Task 01 using C++.

The application implements the major operations required for managing student information, including adding, viewing, searching, updating, and deleting student records. It also incorporates input validation, error handling, and file operations for saving and loading records.

The project demonstrates the practical use of structures, arrays, functions, loops, conditional statements, strings, and file handling. Testing confirmed the expected behavior of the main features under the tested conditions, including validation of incorrect input and retrieval of saved records from the persistent workspace.

Completing this project strengthened my understanding of modular programming, record management, file persistence, and systematic testing. It also provided useful experience in developing, debugging, documenting, and organizing a C++ application.

This project represents an important step in my learning journey as a computer science student and provides a foundation for working on more advanced programming projects in the future.



Project Summary

Items	Details
Organization	Devixo Solutions
Internship Domain	C++ Programming
Task Number	Task 01
Project Title	Student Record Management System
Programming Language	C++
Development Environment	CodeUtility online C++ compiler
Main Source File	main.cpp
Data File	students.txt
Documentation	Project report in PDF format

GITHUB REPOSITORY AND PROJECT DETAILS

GitHub Repository

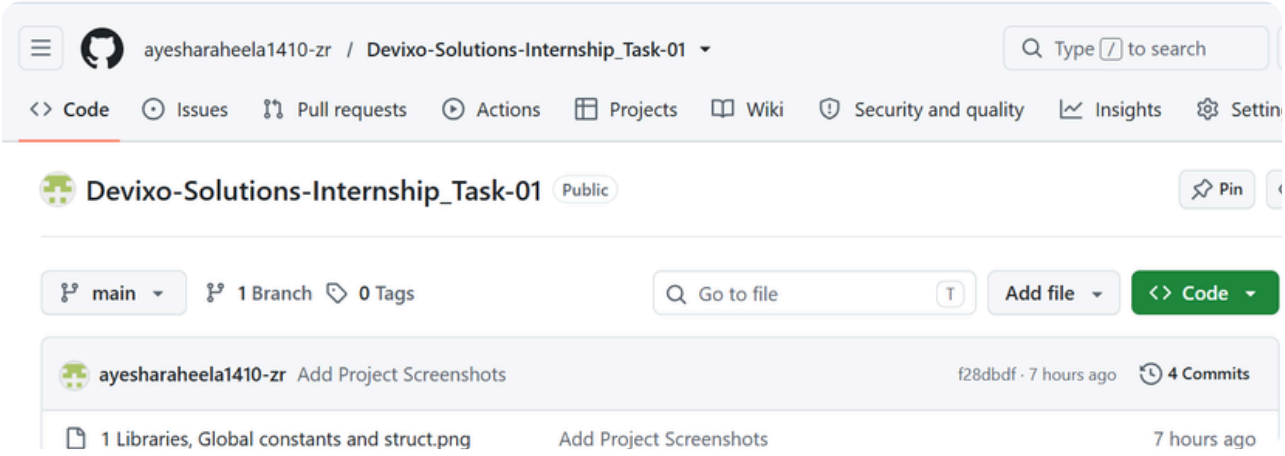
The source code and project documentation are available in the GitHub repository below.

Repository Name: Devixo-Solutions-Internship_Task-01

Repository Link:

https://github.com/ayesharaheela1410-zr/Devixo-Solutions-Internship_Task-01

The repository is intended to provide access to the project source code and README documentation. Refer to the repository for the available project files.



Final Statement



I am grateful for the opportunity to apply my programming knowledge to a practical project. This task helped me strengthen my C++ fundamentals and develop a better understanding of software implementation, testing, and technical documentation.

Submitted by: Ayesha Raheela

End of Report

Thank You!